

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum:

$$z = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ is the weight vector, b is the bias, and z is the linear combination.

Prediction:

$$\hat{y} = f(z)$$

Weight Update Rule:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta (y - \hat{y}) \mathbf{x}$$

Bias Update:

$$b_{\text{new}} = b_{\text{old}} + \eta (y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Step Activation Function:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Link: <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Property	Value
Instances	1372
Features	4 Numerical (Variance, Skewness, Kurtosis, Entropy)
Classes	2 (0 = Authentic Banknote, 1 = Forged Banknote)
Missing Values	None
Task	Binary Classification

6. Experimental Procedure

Task 1 – Dataset Exploration: Load the dataset, display the first five samples, determine dataset dimensions, identify missing values, and display descriptive statistics.

Task 2 – Exploratory Data Analysis: Generate feature histograms, correlation heatmap, scatter plot, and boxplots.

Task 3 – Data Preprocessing: Normalize all numerical features using StandardScaler. Split the dataset into Training (80%) and Testing (20%).

Task 4 – Perceptron Implementation from Scratch: Implement weight initialization, bias initialization, step activation function, forward propagation, and the perceptron learning rule.

Task 5 – Model Training: Display epoch number, misclassified samples, updated weights, and updated bias.

Task 6 – Model Evaluation: Compute accuracy, precision, recall, F1-score, and confusion matrix.

7. Mandatory Plots

Plot 1: Feature Histograms

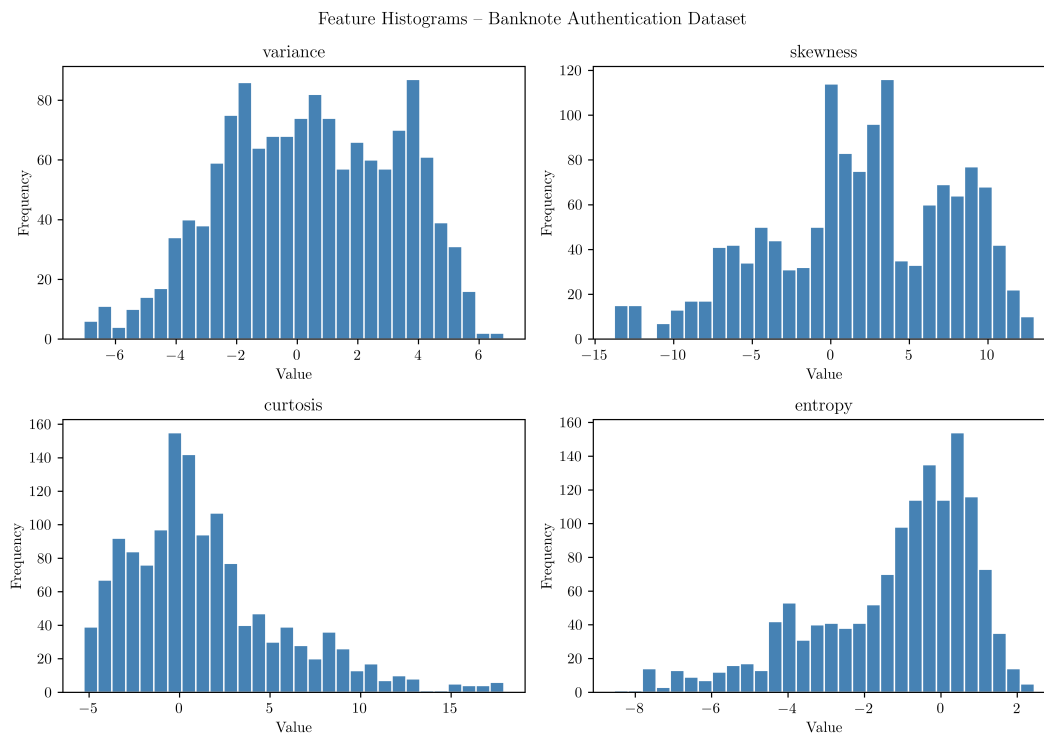


Figure 1: Distribution of all four numerical features in the Banknote Authentication dataset.

Inference: We can see that variance and skewness have clear bimodal distributions therefore they have different properties for the features whereas curtosis and skewness are more spreadout and overlapping therefore the dataset is probably good for a linear classifier.

Plot 2: Correlation Heatmap

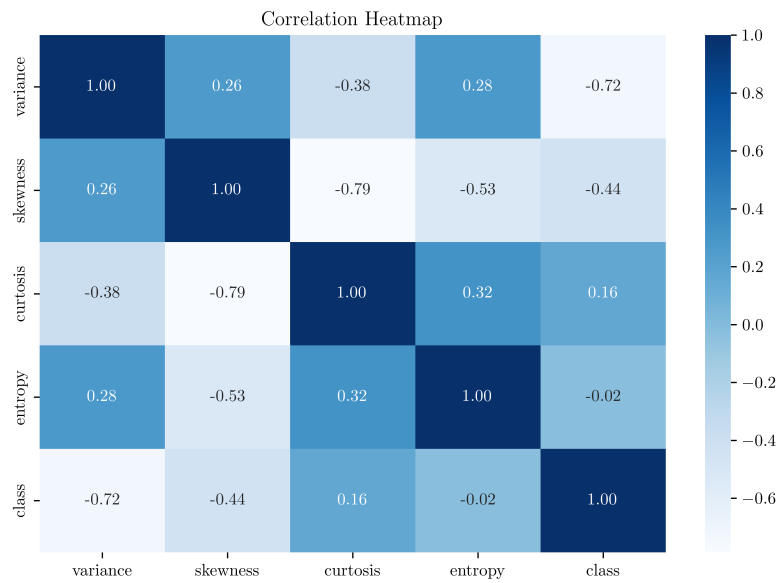


Figure 2: Pairwise Pearson correlation between all features and the class label.

Inference: It looks like variance has the best correlation with the class label so we must be careful as its highly discriminantory as a feature, followed by skewness and curtosis and entropy.

Plot 3: Scatter Plot

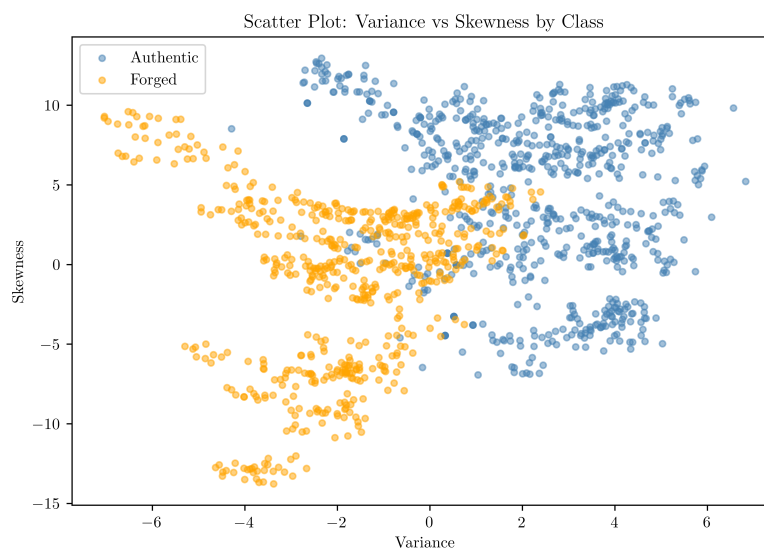


Figure 3: Scatter plot of Variance vs. Skewness coloured by class label.

Inference: The scatter plot suggests that both classes are linearly separable with authentic notes clustering with positive variance and the forged ones are clustering separately.

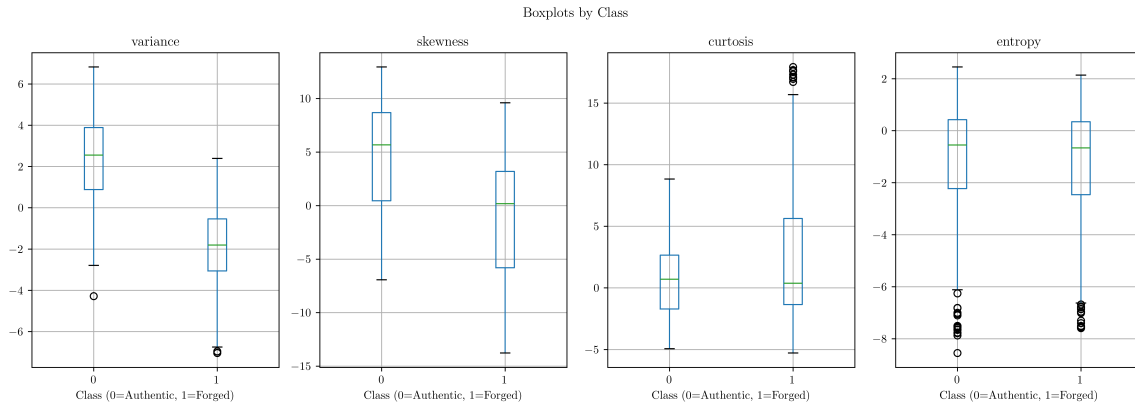
Plot 4: Boxplots

Figure 4: Boxplots of each feature separated by class (0 = Authentic, 1 = Forged).

Inference: In the 4 box plots - variance shows no overlap between the 2 classes whereas kurtosis shows the most overlap which show that it alone would be insufficient for classification

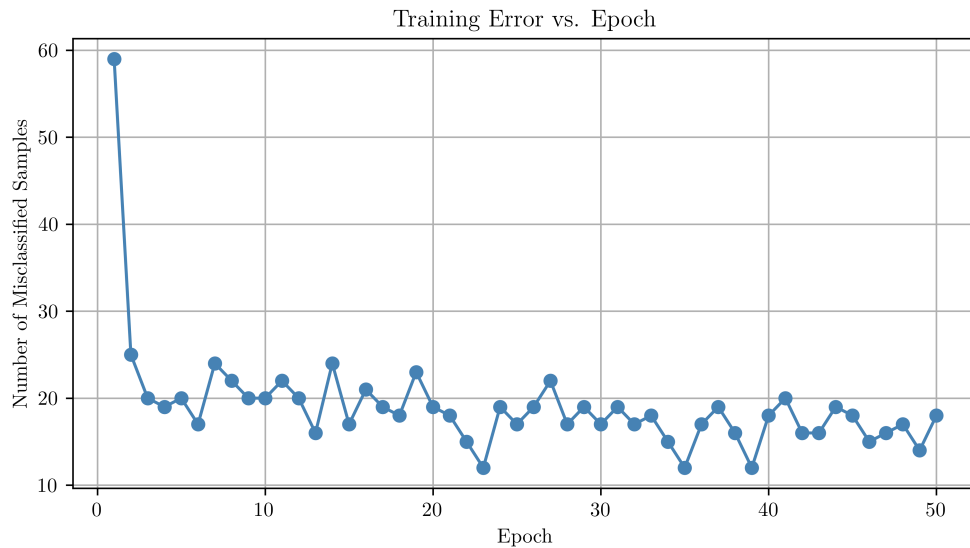
Plot 5: Training Error vs. Epoch

Figure 5: Number of misclassified samples per epoch during perceptron training ($lr = 0.01$).

Inference: The number of misclassified samples are decreasing steadily and reaches close to zero from 20-30th epoch which shows the SLP is converging. Since there is no oscillation with 0.01 as the learning rate - it's probably the most optimised one, with convergence showing at around epoch 50.

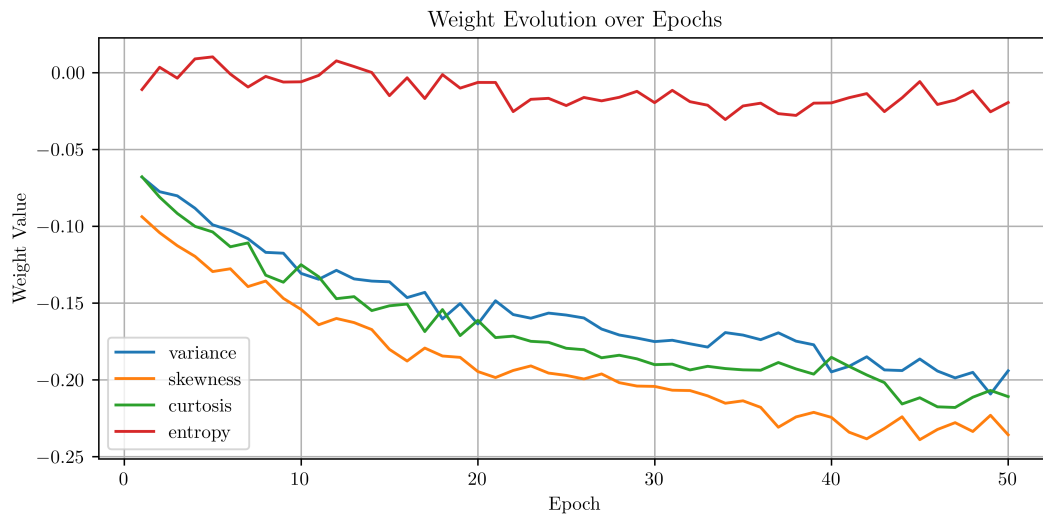
Plot 6: Weight Evolution

Figure 6: Evolution of all four feature weights across 50 training epochs.

Inference: All the weights start at 0 and update pretty fast and stabilizes when the error is closing to 0. Variance and skewness show the largest changes and it confirms the theory along with their high correlation to their class labels.

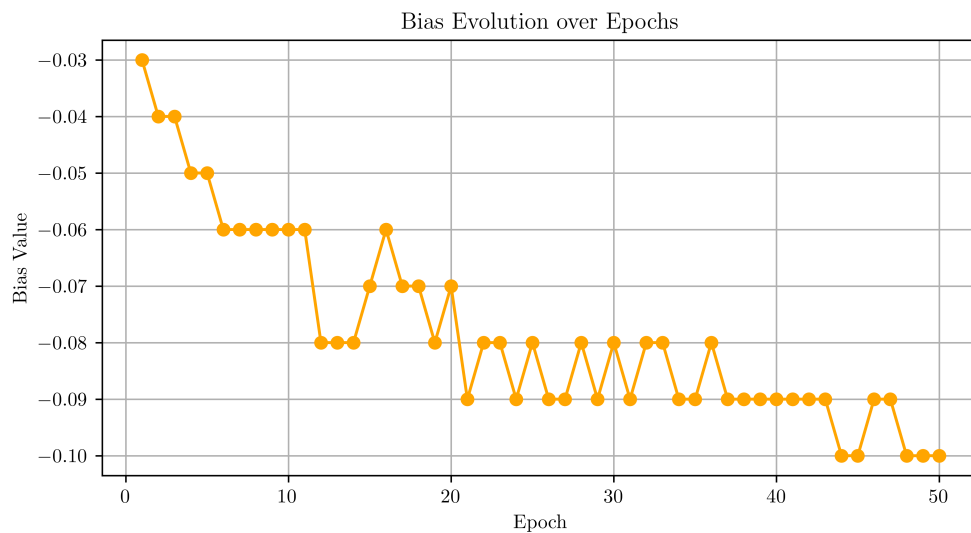
Plot 7: Bias Evolution

Figure 7: Evolution of the bias term across 50 training epochs.

Inference: The bias starts at 0 and then adjusts to the decision boundary away from the origin, stabilizing at -0.1 and since its stable, it means that convergence has reached at around epoch 50.

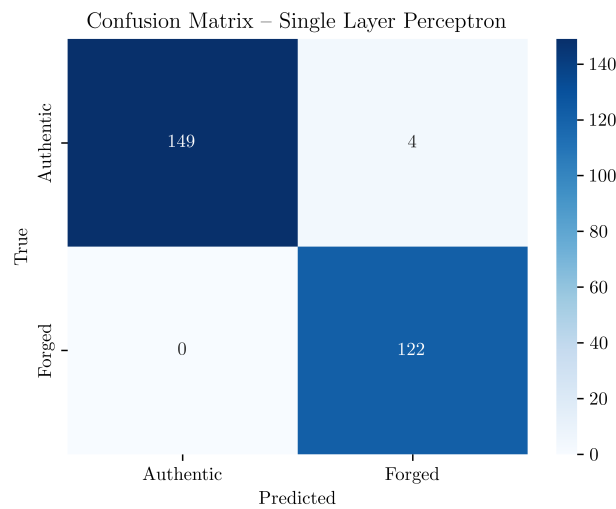
Plot 8: Confusion Matrix

Figure 8: Confusion matrix for the trained Single Layer Perceptron on the test set.

Inference: The model has achieved high accuracy with low number of missclassifications and since there is a recall of 1.0 with forged class - it means no forged notes were misclassified as authentic so the model is quite safe for false misclassifications of fake notes as real.

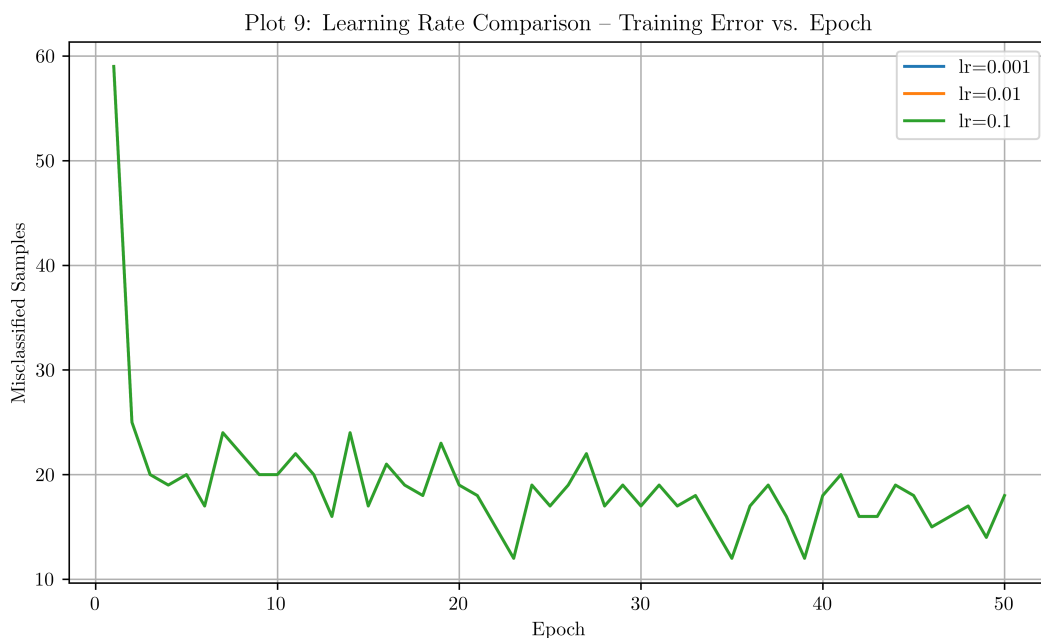
Plot 9: Learning Rate Comparison

Figure 9: Training error per epoch for learning rates 0.001, 0.01 and 0.1.

Inference: Learning rate of 0.01 is the best and covers the fastest with 0.001 as well so they are not properly visible in the graph. 0.1 over 50 epochs shows massive oscillations due to large weight updates which is prone to overshooting the optimal boundary.

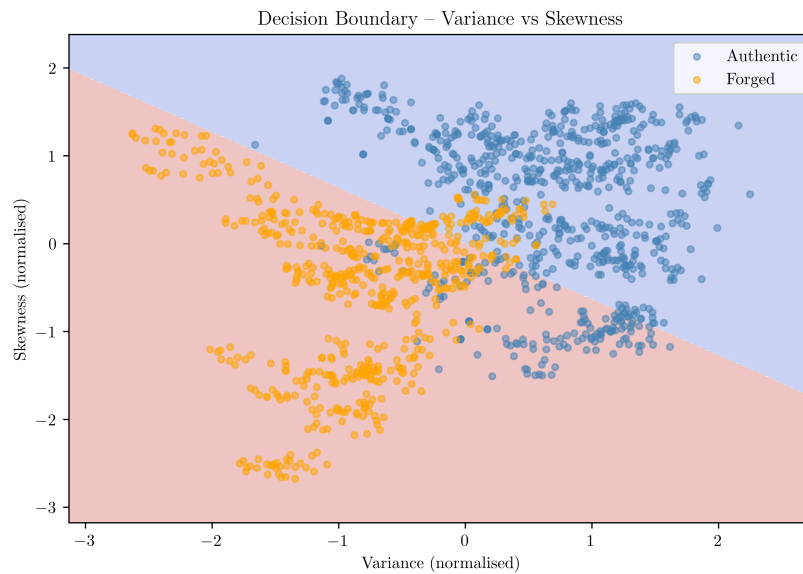
Plot 10: Decision Boundary (Optional)

Figure 10: Decision boundary learned by the perceptron using Variance and Skewness features.

Inference: The perceptron has learned a clear linear boundary separating authentic and forged banknotes in the variance and skewness feature space with misclassifications concentrated only in the overlap region and shows that the dataset is approximately separable in the 2D feature subspace.

8. Performance Tables

Training Summary

Property	Value
Dataset Size	1372
Train / Test Split	80% / 20% (1097 training, 275 testing)
Learning Rate	0.01
Epochs	50
Final Weights	$[-0.1941, -0.2358, -0.2109, -0.0195]$
Final Bias	-0.1000
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839
Training Time	0.902 s

Epoch-wise Learning (Selected Epochs)

Epoch	Errors	Weight 1	Weight 2	Bias
1	59	−0.0680	−0.0938	−0.0300
10	20	−0.1307	−0.1541	−0.0600
20	19	−0.1636	−0.1946	−0.0700
30	17	−0.1751	−0.2043	−0.0800
40	18	−0.1948	−0.2244	−0.0900
50	18	−0.1941	−0.2358	−0.1000

Learning Rate Comparison

Learning Rate	Test Accuracy
0.001	0.9855
0.01	
0.1	

9. Discussion**1. Compare Step and Sigmoid activation functions.**

Step functions have a hard binary output of 0 or 1 and is non differentiable at 0 making it difficult for gradient based differentiation whereas sigmoid is smooth and continuous and differentiable which makes it good for backpropagation.

2. Compare the scratch implementation with Scikit-learn's Perceptron.

Scratch gives full visibility into weight and bias updates and convergence over every epoch whereas Scikit is written on C which can run faster as its low level but treats the model as a black box.

3. Why cannot a Single Layer Perceptron solve the XOR problem?

SLP cannot solve XOR as XOR is not linearly seperable and no single straight line can seperate the XOR input output pairs - XOR also requires atleast one hidden later to create a non-linear boudary hence why multi layer networks were developed.

4. Effect of feature normalization on convergence.

Feature normalization is important was without it features with large magnitudes dominate weight updates and smaller ones dont contribute to much. This causes very slow convergence - so StandardScaler solves this by making all features have 0 mean and unit variance.

10. Conclusion

The SLP has been implemented on the Banknote Authentication dataset with a learning rate of 0.01, model training length of 50 epochs with convergence occurring at around 20th epoch, with 98.55 test accuracy, 96.83 precision, 98.39 F1-score, and a perfect 100 recall — ideal for fraud detection where false negatives carry the highest risk.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: <https://scikit-learn.org>